



浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

Python III: Working with Strings

IB1 1, Lecture 7.1

Dr Duncan MacGregor –

duncan.macgregor@ed.ac.uk

Semester 2, 2025/26

What is a string?



- In programming, just any sequence of characters

"Zhejiang"

"Zhejiang University"

"980k335%janU\$"

Learning objectives

After this lecture, you should be able to:

- Understand the string data type.
- Manipulate strings in Python.
- Understand and use regular expressions to search string objects.

This lecture is a Python reference guide



"They're encyclopedias, Timmy. . . they're an early form of



- The information here will help you write code, but its best to practice – not to memorise.
- You can often achieve the same thing by different means, and you should choose what works best for you (not me!).
- You may find solutions that are not presented here, this is not a comprehensive list.

Outline

1. Strings
2. Regular expressions

str() is a basic data type

- Strings contain textual data written inside quotes (can be single, double or triple)
- **Single quotes**

```
>>> print('Hello world')  
Hello world
```

- Single quotes allow embedded "double quotes"

```
>>> print('He says "Hello world"')  
He says "Hello world"
```

str() is a basic data type

- Strings contain textual data written inside quotes (can be single, double or triple)
- **Double quotes**

```
>>> print("Hello world")  
Hello world
```

- Double quotes allow embedded 'single quotes'

```
>>> print("He says 'Hello world'")  
He says 'Hello world'
```



Adding special characters to str()

- Special characters can be 'escaped' to allow them to be added to a string by inserting a backslash (\) character in front of them

- **Add a tab**

```
>>> print("Hello\tworld")  
Hello world
```

- **Add a newline**

```
>>> print("Hello\nworld")
```

```
--
```

```
world
```


There are lots of special characters!

Escape sequence	Meaning
\n	newline
\t	tab
\\	Backslash (\)
\'	Single quote(')
\"	Double quote (")
\v	vertical tab
...	

- Explore different ways to modify 'Hello world' in your own time

Include special characters in strings

- Strings can be prefixed with a letter 'r' or 'R'
- These raw strings treat backslashes as literal characters.

```
>>>print('Hello world \n and ZJE')
```

```
Hello world
```

```
and ZJE
```

```
>>>print(r'Hello world \n and ZJE')
```

```
Hello world \n and ZJE
```

What do these operations return?

Example 1

```
'Hello\\world'
```

Example 2

```
"\"Hello world\""
```

What do these operations return?

Example 1

```
'Hello\\world'  
Hello\world
```

Example 2

```
"\"Hello world\""  
"Hello world"
```

Recap from week 4: integers and strings

Example 2

```
c = "six"  
d = "seven"
```

What is $c+2*d$? `sixsevenseven`

Example 3

```
i = "6"  
j = "7"
```

What is $i+j$? `67`

Moving between strings and integers

- Each data type can be defined and converted using `str()` and `int()`

```
>>> '1' + 2 + '3'
```

```
'123'
```

```
>>> 1 + 2 + 3
```

```
6
```

```
>>> int('1') + 2
```

```
3
```

- What about this?

```
>>> '1' + str(2)
```

String methods

- Python contains many, many methods for operating on strings
- Slicing can extract part of a string based on their position

```
>>> string = "ABCDEF"
```

```
>>> d = string[3:5]
```

```
DE
```

```
>>> d = string[3:]
```

```
DEF
```

```
>>> d = string[-3:]
```

```
DEF
```

Let's take a break for 5 mins



Outline

1. Strings
2. Regular expressions

Regular expressions

- A regular expression, or regex, is a pattern describing a certain amount of text.
- Regular expressions are a powerful tool for string manipulation.
- We can use regular expressions to:
 - Search a string (search and match)
 - Replace parts of a string (sub)
 - Break strings into smaller pieces (split)

Module `re`

- Before using regular expressions, you must import the library `re`

```
>>> import re
```

To avoid confusion with regular expressions, it's best to use raw strings as `r'expression'`.

```
>>> print('Hello world \n and ZJE')
```

```
Hello world
```

```
and ZJE
```

```
>>> print(r'Hello world \n and ZJE')
```

```
Hello world \n and ZJE
```

re.search()

- Reports whether a string matches a regular expression

```
>>> import re
>>> string = 'abcdef'
>>> re.search(r'cd', string)
<re.Match object; span=(2, 4), match='cd'>
```

- Returns a True/False Boolean depending on whether the string matches the regular expression
- Useful to integrate with 'if' statements:

```
>>> if re.search('cd', string):
    print('yes')
else:
    print('no')
```



a Boolean!

String extraction: `re.findall()`

- To extract the matching strings, we can use `re.findall()`

```
>>> import re
```

```
>>> s = 'my 2 favourite numbers are 27 and 69'
```

```
>>> number1 = re.findall(r'[0-9]+', s)
```

```
>>> print(number1)
```

```
['2', '27', '69']
```



a list of substrings

[0-9]+ to find
digits, and
match multiple
hits

Regular expressions: Rob's quick guide

.	matches any character
+	repeats the previous element one or more
*	repeats the previous element zero or more
^	match the beginning of the line
\$	match the end of the line
	match either

[a-z]	set of characters includes the range a-z
[a-zA-Z]	set of characters including lower and upper case
[0-9]	matches any decimal digit in this range
\s	any whitespace character
\S	any non-whitespace character
\d	any decimal digit

etc, etc, etc...

Example regular expressing: do these match?

```
re.search(r'[0-9][a-zA-Z]', '11Edderston')
```

```
re.search(r'[0-9][a-z]', '11Edderston')
```

```
re.search(r'^\d\d', '11Edderston')
```

```
re.search(r'$\d\d', '11Edderston')
```

Example regular expressing: do these match?

```
re.search('[0-9][a-zA-Z]', '11Edderston') Yes
```

```
re.search('[0-9][a-z]', '11Edderston') No, requires lower case
```

```
re.search('^\\d\\d', '11Edderston') Yes
```

```
re.search('\\$\\d\\d', '11Edderston') No, requires numbers at the end
```


String extraction continued: `re.findall()`

- Parentheses can be used to define start and end of extracted strings

```
>>> import re
```

```
>>> x = 'From robot@zju.edu.cn Feb 10:38:06 2025'
```

```
>>> y = re.findall(r'^From (\S+@\S+)', x)
```

```
>>> print (y)
```

```
['robot@zju.edu.cn']
```

Starting at the beginning of the line, look for the string 'From'

Extract the non-blank characters

At least one non-whitespace character

Look through the string until you find at sign

Extract the non-blank characters

```
^From (\S+@\S+)
```

String extraction continued:

`re.findall()`

- How to extract the login name?

```
>>> import re
```

```
>>> x = 'From robot@zju.edu.cn Feb 10:38:06 2025'
```

String extraction continued:

`re.findall()`

- How to extract the login name?

```
>>> import re
>>> x = 'From robot@zju.edu.cn Feb 10:38:06 2025'
>>> y = re.findall(r'(\S+)@', x)
>>> y = re.findall(r'([^\s]+)@', x)
```

`(\S+)@`

Extract the non-whitespace characters

At least one non-whitespace character

`([^\s]+)@`

Look through the string until you find at at-sign

Match a non-blank character

Greedy matching

- The repeat characters (* and +) push outward in both directions (greedy) to match the **longest** possible string.

```
>>> import re
>>> x = 'From: Using the :character'
>>> y = re.findall(r'^F.+:', x)
>>> print(y)
['From: Using the :']
```

First character is F

^F.+:

One or more characters

Last character in the match is :

'From: Using the :character'



Non-greedy matching

- Not all regular expression repeat codes are greedy!
- If you add a ? character the + and * chill out a bit

```
>>> import re
>>> x = 'From: Using the :character'
>>> y = re.findall(r'^F.+?:', x)
>>> print(y)
['From:']
```

First character is F

One or more characters but not greedy

Last character in the match is :

^F.+?:

'From: Using the :character'



Matching examples

Example 1

```
x = '817 East Haizhou Road'  
y = re.findall(r'(\S+)\s+R', x)
```

Example 2

```
x = '817 East Haizhou Road'  
y = re.findall(r'^(\S+)\s+E', x)
```

Matching examples

Example 1

```
x = '817 East Haizhou Road'  
y = re.findall(r'(\S+)\s+R', x)
```

Haizhou

Example 2

```
x = '817 East Haizhou Road'  
y = re.findall(r'^(\S+)\s+E', x)
```

817

Split strings

- **re.split(pattern, string)** can be used to split strings based on regular expression

```
>>> x = 'a b c d e'
>>> x.split(' ')
['a', 'b', 'c', 'd', 'e']
>>> re.split(r'\s+', x)
['a', 'b', 'c', 'd', 'e']
>>> z = 'a ,b ; c ;d,e'
>>> re.split(r'\s; ', z)
['a ,b', ' c', 'd,e']
```


Substitute strings

- **re.sub(pattern, replacement, string)** replicates all matches of pattern in string with replacement
- The initial string is **not** modified

<code>result = re.sub(r'abc', '', input)</code>	delete pattern abc
<code>result = re.sub(r'abc', 'def', input)</code>	replace abc->def
<code>result = re.sub(r'\s+', ' ', input)</code>	eliminate duplicate whitespaces

Learning objectives

Now, you should be able to:

- Understand the string data type.
- Manipulate strings in Python.
- Understand and use regular expressions to search string objects.

We will practice these operations further in this week's practical.





浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

Python III: Working with Strings

Any questions?

IB1 1, Lecture 7.1

Dr Duncan MacGregor –

duncan.macgregor@ed.ac.uk

Semester 2, 2025/26